Polynomial Activations and Projective Normalization: An Empirical Study of What Makes Deep Fully-Connected Networks Trainable

Xiuyi Yang
fwinlee@gmail.com

October 13, 2026

Code, raw results and paper sources: https://github.com/lfwin/Polynomial-Activation-and-projective-normalization

Abstract

Two ingredients decide whether a deep network trains: the *nonlinearity* that breaks linearity, and the *normalization* that keeps its signal in range. This paper studies both, and their interaction, through the lens of a single construction: polynomial activations paired with *projective* (ℓ_2) normalization. We take as our starting point a ten-layer fully-connected network whose activation is the projective polynomial $f(z) = z + \lambda z^3$, whose output is projected onto the unit sphere. We separate the two mechanisms that this construction entangles — the *polynomial nonlinearity* and the *projective normalization* — and evaluate each against the activations and normalizers that current large language models actually use (ReLU, GELU, SiLU, the gated SwiGLU and GeGLU; BatchNorm, LayerNorm, RMSNorm), plus classical controls (tanh, identity) and learnable polynomial bases (Legendre, Chebyshev, KAN-style cubic B-splines). Experiments span MNIST and eight synthetic datasets, depths from 2 to 50 layers, polynomial orders 2–5, $\lambda \in [10^{-3}, 10^{-1}]$, and five seeds. Three results emerge, all of them about the polynomial–normalization pair rather than about either alone. First, the polynomial nonlinearity is *not* competitive on raw accuracy: it trails GELU by 3.86 points at equal parameter count. Second, the projective normalization recovers 1.59 of those points with *zero* learned parameters, and it does so by a mechanism we can measure — it pins the per-layer activation norm at unity ($1\times$ its initial value) where ReLU's decays to $0.048\times$ and a gated unit's collapses to 0. Third, this same normalization is what makes depth reachable: the projected polynomial tracks ReLU to within 2.14 points at depth ten and 0.29 points at depth thirty, whereas an unnormalized polynomial collapses to chance at depth 30; and a gated unit without normalization fails outright while the same unit trains at 97.98% once normalization is added. We report the failure boundary honestly: at depth 50 both designs collapse to $\approx$10.28%, so projective normalization buys depth, not unbounded depth.

1 Introduction

Depth is what makes a neural network expressive [Hornik et al., 1989, Cybenko, 1989, Mhaskar and Poggio, 2016, Telgarsky, 2016], and it is also what makes it hard to train [Glorot and Bengio, 2010, He et al., 2015, Poole et al., 2016, Schoenholz et al., 2017]. Two families of design decisions carry most of the weight in keeping a deep stack trainable: the *nonlinearity* applied between affine maps, and the *normalization* applied to the activations. The modern defaults are well known — ReLU [Glorot et al., 2011, Maas et al., 2013] or GELU [Hendrycks and Gimpel, 2016] for the

nonlinearity, and BatchNorm [Ioffe and Szegedy, 2015] or LayerNorm [Ba et al., 2016] for the normalization — but the space of alternatives is far from exhausted, and the two decisions interact in ways that a single-axis ablation cannot expose.

This paper takes as its starting point a specific ten-layer network in which both decisions are made *together*, by a single map:

$$\pi(f(z)) = \frac{f(z)}{\|f(z)\|_2}, \qquad f(z) = z + \lambda z^3, \tag{1}$$

where z is the pre-activation of a layer and the ℓ_2 norm is taken over the feature dimension of each sample. We call this a *projective polynomial* activation: *polynomial* because the nonlinearity is the truncated cubic $z + \lambda z^3$ rather than a saturating or rectifying function, and *projective* because the output is pushed onto the unit sphere, so each sample's feature vector is identified with a point of projective space rather than of Euclidean space.

The construction is attractive on several grounds. It is parameter-free (no learned scale or shift), it is batch-independent (unlike BatchNorm, it does not couple samples within a mini-batch), it has no train/eval discrepancy, and it provably bounds the activation magnitude, which in a deep stack bounds the growth of the forward signal and, by the chain rule, the growth of the backward signal. It also inherits the projective-geometric intuition that motivates a body of work on the algebraic geometry of linear networks, where the loss landscape of a deep stack is studied through the projective geometry of its critical points [Yang, 2021, Saxe et al., 2014, Dauphin et al., 2014].

The difficulty with Eq. (1) as a scientific object is that it bundles four separate claims:

(C1) a polynomial nonlinearity is a good nonlinearity;

(C2) an odd, unbounded-but-truncated polynomial is preferable to a rectifying or saturating one;

(C3) normalizing activations helps, and the specific normalization ℓ_2 projection is preferable to centering-and-scaling alternatives;

(C4) *depth* is where these differences matter, so the comparison must be made at depths where naive networks fail.

Testing Eq. (1) as a monolith therefore tells us whether *this* design works, not *which* of its ingredients is responsible. The central methodological move of this paper is to pull the construction apart: we implement the polynomial and the projection as independent, independently switchable components, and then vary each against a broad reference set.

What we compare. We organize the study around four reference families.

- *Pointwise activations.* Identity (linear control), the classical saturating tanh, and the three non-saturating activations that current frontier language models actually deploy: ReLU [Glorot et al., 2011], GELU [Hendrycks and Gimpel, 2016] and SiLU [Elfwing et al., 2018]. We deliberately do not re-audit the historical zoo (sigmoid, LeakyReLU, ELU, SELU, Softplus, Mish, sinusoidal) [Maas et al., 2013, Clevert et al., 2016, Klambauer et al., 2017, Misra, 2020, Sitzmann et al., 2020]; none of those appears in a current large model, and our question is where a polynomial activation stands relative to what is deployed, not how the zoo ranks.

- *Gated linear units.* SwiGLU and GeGLU [Shazeer, 2020, Dauphin et al., 2017] — the shapes used in current large language models — plus a polynomial-gated PolyGLU, which is the natural way to carry the paper's cubic into a gate. We include gating because "the activation

question" in modern practice is often really a question about gating. Gated units contain two matrices per layer instead of one, so we match parameter count against the pointwise networks rather than comparing differently sized models.

- *Learnable polynomial activations.* A Legendre series, a Chebyshev series, and a KAN-style cubic B-spline [Liu et al., 2024]. If a fixed cubic is useful, the natural question is whether letting the polynomial be learned improves on it; this family answers that question directly.
- *Normalization layers.* BatchNorm [Ioffe and Szegedy, 2015], LayerNorm [Ba et al., 2016], RMSNorm [Zhang and Sennrich, 2019], and the parameter-free ℓ_2 projection, each placed both before and after the nonlinearity, and each crossed with both a rectified and a polynomial activation.

How we compare. All comparisons are made inside a single harness that isolates one factor at a time, with the source network of Eq. (1) reproduced bit-for-bit as the reference configuration (Sec. 4). We vary depth from 2 to 50 layers, polynomial order from 2 to 5, the polynomial coefficient λ over three decades, the ℓ_2-projection placement, the presence of a learnable activation, and the random seed. Alongside the ten-class MNIST task we use eight synthetic datasets, including ones that are deliberately hostile to a polynomial activation: a parity task, where the label depends on a high-order interaction of binary features, and a high-dimensional Gaussian task, where a low-dimensional signal is embedded in a 50-dimensional ambient space.

Findings. Our main conclusions are the following.

- The projective polynomial network is *not* competitive at equal parameter count: at depth ten the bare cubic $z + \lambda z^3$ lands almost four accuracy points *below* ReLU, GELU and SiLU, and the parameter-free ℓ_2 projection recovers a little over half of that deficit — enough to bring the polynomial within two and a half points of the modern activations, but not enough to match them. Measured against the reference network's own depth, the polynomial is a reasonable but strictly dominated nonlinearity, and the paper's honest claim is about *what the projection does*, not about the polynomial beating ReLU.
- The performance of Eq. (1) comes *predominantly from the projection, not the polynomial*: holding the polynomial fixed and removing the ℓ_2 projection costs accuracy comparable to removing the nonlinearity's advantage over ReLU entirely, whereas holding the projection fixed and varying the polynomial changes accuracy only modestly.
- At depth, the ℓ_2 projection *substitutes for* BatchNorm and LayerNorm: it recovers most of their benefit while using zero parameters and no batch statistics.
- Making the polynomial learnable (Legendre, Chebyshev, B-spline) yields no consistent gain over the fixed cubic at equal depth and width: the learned bases are more expensive and no more accurate.
- Gated units (SwiGLU, GeGLU) require normalization to train at depth at all, and once parameters are matched and normalization is present they are in the same band as the best pointwise activations; the large margin sometimes reported for gated units is a parameter-count effect, not an activation effect.

- At depth ten on MNIST, a gated unit with no normalization fails to train at all, sitting at chance accuracy. The same unit trains once normalization is restored. This is the clearest single piece of evidence in our study that, for these architectures, *normalization is not an optional refinement but a precondition for depth.*

The remainder of the paper is organized as follows. Sec. 2 positions the work. Sec. 3 defines the model and the activation and normalization families precisely. Sec. 4 describes the experimental protocol. Sec. 5 reports results, and Sec. 6 discusses what they do and do not license. Sec. 7 concludes.

2 Related Work

Activation functions. The sigmoid and tanh activations of early multilayer perceptrons saturate, and their gradients vanish exponentially with depth, which is the classical explanation for why deep stacks were impractical before the rectifier era [Glorot and Bengio, 2010, Glorot et al., 2011]. ReLU removes saturation on the positive side and yields sparse activations [Glorot et al., 2011]; its variants (LeakyReLU [Maas et al., 2013], ELU [Clevert et al., 2016], SELU [Klambauer et al., 2017], PReLU [He et al., 2015]) address the dying-unit and negative-saturation issues at some cost in simplicity. Smooth alternatives that interpolate between the identity on the positive side and a soft rectifier on the negative side — GELU [Hendrycks and Gimpel, 2016] and SiLU [Elfwing et al., 2018, Ramachandran et al., 2017] — are now the default in most large-scale vision and language models, and are the activations our polynomial most directly competes with. We therefore restrict the pointwise comparison to this deployed set (ReLU, GELU, SiLU) plus tanh and the identity control, rather than re-auditing the historical variants.

Polynomial and learnable activations. Polynomial activations have a long history: the higher-order units of early connectionist models, and more recently the observation that a polynomial network of low degree can compute the *exact* product of its inputs, which a ReLU network can only approximate with growing width [Hornik et al., 1989, Cybenko, 1989, Izhikevich, 2007, Neyshabur et al., 2015]. Learnable activation functions — PReLU [He et al., 2015] and its family, "maxout" networks, and the recent Kolmogorov–Arnold network (KAN) whose edges carry learnable spline univariates [Liu et al., 2024] — move the nonlinearity into the parameter set. Our study includes both ends of this axis: a *fixed* cubic, and *learnable* polynomials in the Legendre, Chebyshev and cubic B-spline bases.

Gated linear units. The gated linear unit multiplies a linear projection of the input by a gate-shaped version of it [Dauphin et al., 2017]. Substituting SiLU for the sigmoid in this construction yields SwiGLU, which is standard in current transformer feed-forward blocks; GeGLU is the analogous construction with a GELU gate [Shazeer, 2020]. A recurring methodological problem in this literature is that gated units carry roughly 3/2 times the parameters of the ungated layer they replace, so a gain in accuracy can reflect capacity rather than gating. We therefore match parameter count explicitly and report an unmatched-width control.

Normalization layers. BatchNorm normalizes each feature using mini-batch statistics, coupling samples and introducing a train/eval discrepancy resolved by running statistics [Ioffe and Szegedy, 2015, Santurkar et al., 2018, Bjorck et al., 2018]. LayerNorm normalizes per sample instead, removing the batch coupling [Ba et al., 2016]; RMSNorm drops the mean-centering and is standard in current

transformers [Zhang and Sennrich, 2019]. Weight normalization [Salimans and Kingma, 2016] and the ℓ_2 normalization of weight vectors are the parameter-free relatives of this family. Other cheap scale-control techniques operate on gradients rather than on activations [Yong et al., 2020]. The ℓ_2 projection in Eq. (1) belongs to that parameter-free subfamily, and the question we study — whether activation *projection* can substitute for statistical normalization — is the natural one to ask of it.

Geometry of deep networks. A parallel literature studies deep networks through the geometry of their loss landscape: the role of saddle points in the training of linear and residual networks [Saxe et al., 2014, Dauphin et al., 2014, Goodfellow et al., 2015], the projective geometry of the critical locus of a linear network's gradient equations [Yang, 2021], and the way normalization and residual connections reshape that locus [He et al., 2016, Balduzzi et al., 2017]. A related strand concerns the implicit bias of gradient descent on separable data, which bears directly on our nearly-linearly-separable MNIST benchmark [Soudry et al., 2018]. Our construction is the architectural analogue of that reading — identifying a sample's activation with a point of projective space — and our experimental question, which component of the construction carries the benefit, is the empirical counterpart of asking which component of the geometry is responsible for trainability. We note explicitly that we do *not* claim a theorem here: we treat the projective view as motivation for the architecture, and we restrict our claims to the measured behavior of the network.

Empirical comparisons and reproducibility. Ablation studies of activations are notoriously sensitive to uncontrolled factors: parameter count, initialization, learning rate, and depth interact with the activation choice, and a single-seed comparison at one depth is weak evidence. Our protocol varies each factor independently, holds the initialization and optimizer fixed, matches parameter count across architecturally different families, and repeats the headline comparison over several random seeds. We also report per-configuration wall-clock cost, since one of our findings (the poor conditioning of the unnormalized gated unit) is visible only in the cost profile.

3 Method

3.1 Reference architecture

The reference network is the source network of this study: a depth-L fully-connected classifier with L linear layers, the first $L-1$ of which are followed by the same activation–normalization block, and the last of which produces class logits without either. Writing d_0 for the input dimension (784 for MNIST), H for the hidden width, and C for the number of classes (10 for MNIST), the network is

$$h^{(0)} = x, \tag{2}$$

$$h^{(\ell)} = \pi(f(W_\ell h^{(\ell-1)} + b_\ell)), \qquad \ell = 1, \dots, L-1, \tag{3}$$

$$\hat{y} = W_L h^{(L-1)} + b_L, \tag{4}$$

with $x \in \mathbb{R}^{d_0}$ flattened from the input, $W_1 \in \mathbb{R}^{H \times d_0}$, $W_\ell \in \mathbb{R}^{H \times H}$ for $2 \le \ell \le L-1$, and $W_L \in \mathbb{R}^{C \times H}$. All weights are initialized with Xavier-uniform and all biases to zero. The block $h \mapsto \pi(f(Wh + b))$ is the object of study; both f and π are independently selectable in our implementation, and the reference configuration is $f(z) = z + \lambda z^3$ with $\lambda = 10^{-2}$, $\pi(y) = y/\|y\|_2$.

3.2 The projective polynomial block

The reference activation is

$$f(z) = \mathrm{clamp}(z, [-c, c]) + \lambda\, \mathrm{clamp}(z, [-c, c])^3, \qquad c = 10, \tag{5}$$

applied elementwise, followed by the per-sample projection

$$\pi(y) = \frac{y}{\max\{\|y\|_2, \epsilon\}}, \qquad \epsilon = 10^{-8}, \tag{6}$$

where the norm runs over the feature axis, so that $\pi(y)$ is a point of the unit sphere and the layer map is invariant to the radial scale of y. We emphasize three properties that motivate this choice and that our ablations probe separately.

- *Bounded growth.* The clamp bounds $|f(z)| \le c + \lambda c^3$, so the forward signal cannot blow up; the projection then bounds the *norm* of each hidden activation at exactly 1.
- *No statistics, no state.* π has no parameters and no running buffers. Unlike BatchNorm [Ioffe and Szegedy, 2015] it does not couple samples, and unlike both BatchNorm and LayerNorm [Ba et al., 2016] it has no learned affine ("scale and shift") parameters, so it changes the geometry of the representation without adding a single degree of freedom.
- *Radial invariance.* Because π identifies y with $y/\|y\|$, the block discards the radial direction at every layer, which is the sense in which the construction is projective.

We separate the two components in the implementation: `-activation poly` gives Eq. (5) *without* the projection, `-norm l2` gives Eq. (6) *without* the polynomial, and `-activation poly_proj` gives the reference block of Eq. (1). This factorization is what allows the same harness to attribute performance to the polynomial, to the projection, or to their combination.

Placement. Normalization can be applied before or after the nonlinearity. Write the block as $\pi \circ f$ ("post") or $f \circ \pi$ ("pre"). The reference construction is post-normalization; we report both, since pre-normalization is standard in transformers while post-normalization was the original BatchNorm convention.

3.3 Activation families

Let z denote the pre-activation. All pointwise activations are applied elementwise.

Pointwise. Identity (a linear control), the classical saturating $\tanh(z)$, and the three non-saturating activations that current large language models use in practice: $\mathrm{ReLU}(z) = \max(0, z)$ [Glorot et al., 2011], GELU $z\Phi(z)$ with Φ the standard normal CDF ("exact", not the tanh approximation) [Hendrycks and Gimpel, 2016], and SiLU $z\sigma(z)$ [Elfwing et al., 2018]. We deliberately restrict the pointwise family to these, rather than sweeping the many historical variants (LeakyReLU, ELU, SELU, Softplus, Mish, sinusoidal), because none of those appears in a current frontier language model; the study is meant to state where a polynomial activation stands relative to what is actually deployed, not to re-audit the activation-function zoo. The identity map is retained only as a control that makes the contribution of nonlinearity itself measurable.

Polynomial. The fixed truncated cubic of Eq. (5), with the order and coefficient treated as hyperparameters, and three *learnable* polynomial activations [Liu et al., 2024]:

$$f_{\mathrm{Leg}}(z) = g\sum_{k=0}^{K} a_k L_k\left(\frac{\tilde{z}}{B}\right), \qquad f_{\mathrm{Cheb}}(z) = g\sum_{k=0}^{K} a_k T_k\left(\frac{\tilde{z}}{B}\right), \qquad \tilde{z} = \mathrm{clamp}\left(\frac{z}{s}, [-B, B]\right), \tag{7}$$

where L_k and T_k are the Legendre and Chebyshev polynomials of degree k, s and B fix the normalization window, and (a_k) together with the gain g are learned. We initialize $a_1 = 1$, $a_{k\neq 1} = 0$, $g = 1$, so every learnable activation *starts as the identity map* and must justify any departure from it. For the spline variant we use the KAN construction

$$f_{\mathrm{spline}}(z) = w_{\mathrm{base}}\ \mathrm{SiLU}(z) + w_{\mathrm{spline}}\sum_j c_j N_j(\mathrm{clamp}(z, [-B, B])), \tag{8}$$

with N_j cubic B-spline basis functions on a uniform grid and control points c_j initialized to zero, so the activation also starts at $w_{\mathrm{base}}\,\mathrm{SiLU}(z)$ — the KAN initialization.

Gated units. A gated unit replaces the pointwise map by

$$\mathrm{GLU}_\sigma(Wx) = (W_v x) \odot \sigma(W_g x), \tag{9}$$

with two matrices W_v, W_g and a gate nonlinearity σ. We use the two gates that current language models deploy [Shazeer, 2020] — $\sigma = \mathrm{SiLU}$ (SwiGLU, as in Llama and PaLM) and $\sigma = \mathrm{GELU}$ (GeGLU) — and, as the paper's polynomial counterpart, $\sigma = z + \lambda z^3$ (PolyGLU). Gated units with historical gate nonlinearities (ReGLU, SELU-GLU, Mish-GLU, Tanh-GLU) and the fully degenerate *bilinear* unit ($\sigma = \mathrm{Id}$) are omitted for the same reason as the historical pointwise activations: they are not what is deployed, and including them would broaden the failure-mode survey without changing the comparison the paper is about.

Because Eq. (9) carries about twice the parameters of a pointwise block at the same width, a pointwise-versus-gated comparison at equal width confounds the activation with capacity. We therefore report gated results at a width chosen so that the *total* parameter count matches the pointwise network, and we additionally report pointwise networks at that reduced width as a control. Concretely, for a depth-L network with input dimension d_0 and C classes, a pointwise network at width W_e has $d_0 W_e + (L-2)W_e^2 + CW_e + C$ weights, while a gated network at width W_g has $2d_0 W_g + 2(L-2)W_g^2 + CW_g + C$; we choose $W_g = 82$ to match the pointwise width $W_e = 128$ as closely as an integer width allows at $L = 10$, $d_0 = 784$, $C = 10$. At these settings the two blocks have 233,866 and 238,466 parameters respectively, a residual gap of 2.0%; *no integer width removes this gap*, because the gated block carries a second $d_0 W_g$ term and a second $(L-2)W_g$ multiplier, which is not an affine function of the width. We report this gap rather than hiding it, and we also report pointwise networks at the reduced width $W_g = 82$ as a control, so that the activation effect can be read off the matched-capacity rows as well as the matched-width rows.

3.4 Normalization families

We compare five normalization choices, all applied to the hidden activation with the feature axis as the normalization axis.

- *None*, the unnormalized control.
- *Projection* (π): Eq. (6), no parameters.

- *BatchNorm*: per-feature standardization by mini-batch mean and variance, with learned scale and shift and running statistics for evaluation.
- *LayerNorm*: per-sample standardization over the feature axis, with learned scale and shift.
- *RMSNorm*: per-sample root-mean-square scaling, with a learned scale but no mean subtraction and no shift.

This choice makes the comparison informative in a specific way: BatchNorm and LayerNorm both add a learned affine map after normalizing, while projection and RMSNorm (essentially) do not, so we can separate the effect of *normalizing* from the effect of *re-parameterizing with a learned affine map*.

3.5 Diagnostics

For each run we record, in addition to loss and accuracy curves, the per-layer gradient norms and the per-layer activation norms at initialization and after training, on a fixed mini-batch. These let us distinguish runs that differ in accuracy because of optimization (gradient profile) from runs that differ because of representation scale (activation profile), and in particular they make visible the collapse of the unnormalized gated unit reported in Sec. 5.

4 Experimental Setup

4.1 Single harness, reproducible reference

All results come from one implementation, which reproduces the reference network exactly. We verified this by running the reference configuration (`-activation poly_proj`, i.e. Eq. (1)) for two epochs and confirming that the per-epoch train/test accuracies agree with the standalone reference script to the last reported digit. The fidelity run's values are 87.15% / 90.51% after epoch one and 91.52% / 92.16% after epoch two (train / test); note that the reference script's default epoch count is twenty, and the cosine schedule has period equal to the epoch budget, so the two-epoch fidelity check is compared against the reference script run with the same two-epoch budget. Because the harness is a strict superset of the reference script, every number in this paper is comparable to every other number, and the reference configuration appears in every table as the baseline row: with its own twenty-epoch budget it reaches 98.26% train and 96.86% test accuracy in 2.0 s per epoch (40 s total).

4.2 Data

MNIST. 60,000 training and 10,000 test grayscale images [LeCun et al., 1998], 28×28, flattened to 784 dimensions with the raw pixel scale in $[0, 1]$ (only `ToTensor`, no mean-variance standardization — the reference pipeline uses none, and adding one would change the effective scale of the activation, which is precisely one of the axes under study). Ten classes.

Synthetic. Eight generator families, each a deterministic function of a seed, with the test split drawn from the same generator at a different seed so that the two splits are independent samples of the same distribution rather than a split of one finite sample.

- *rings* (5 classes, 2-D): concentric noisy circles; requires the network to use the radius, i.e. a *quadratic* function of the input. Directly relevant to whether a cubic activation helps on a polynomial task.
- *spiral* (5 classes, 2-D): the classical interleaved spiral; requires a strongly nonlinear, non-monotone decision boundary.
- *blobs* (5 classes, 2-D): well-separated Gaussians; an easy control that should saturate for every activation and therefore calibrates the other results.
- *xor* (2 classes, 2-D): parity of the coordinate signs; the minimal case where an additive (linear-plus-pointwise) model must build a product.
- *checker* (2 classes, 2-D): a 4×4 checkerboard; high-frequency and piecewise constant, the regime where a smooth activation is expected to have difficulty.
- *cubes* (2 classes, 3-D): parity of three dichotomized coordinates, i.e. a *third-order* interaction, matched to the order of the reference cubic.
- *parity* (2 classes, 10-D): parity of ten binary features. The label depends on the tenth-order interaction, so any model that can only form low-order or additive functions must fail; this is our stress test for whether a degree-3 activation's advantage is about order or about scale.
- *gauss_highdim* (5 classes, 50-D): a five-class problem living on a two-dimensional subspace, embedded in fifty dimensions with additive noise. Tests whether the projection's scale control helps when most input directions are noise.

For the synthetic experiments the input is standardized using *training* split statistics only. We use 4000 training and 800 test samples per family.

4.3 Models and hyperparameters

Unless stated otherwise, the configuration is depth $L = 10$ linear layers, hidden width $H = 128$, AdamW [Loshchilov and Hutter, 2019] with learning rate 10^{-3} (following Adam [Kingma and Ba, 2015]) and weight decay 10^{-4}, cosine-annealed over the run, gradient-norm clipping at 5.0 [Pascanu et al., 2013], batch size 128, and 10 epochs. The learning rate, weight decay, clipping threshold, batch size and scheduler all match the reference script, so the reference row of each table is a faithful baseline rather than a re-tuned one. The synthetic experiments use the shallower, smaller setting requested for that part of the study: $L = 3$, $H = 64$, learning rate 3×10^{-3}, batch size 64, 60 epochs. The headline comparison is repeated over five seeds (0–4) and reported as mean $\pm$ standard deviation; all other tables use the single reference seed 42.

4.4 Sweeps

Table 1 lists the axes. Three design choices deserve comment. First, wherever an architectural family changes the parameter count — gated units, learnable activations, the spline basis — we report the matched parameter count alongside the accuracy, and we include a width-controlled pointwise baseline, so that no conclusion rests on an accidental capacity difference. Second, following the same deployability criterion that fixed the activation list, we restrict the sweep to activations that a current frontier language model actually uses, plus the polynomial family that is the subject of the paper; the historical activation zoo is not re-audited. Third, the depth sweep includes 30 and

Table 1: Sweeps. Each row varies one factor of the reference configuration while holding the rest at the reference values. L = depth, H = width, d = polynomial order, λ = polynomial coefficient, pos = placement of the normalization relative to the nonlinearity.

Axis	Values	Fixed	Purpose
Activation (pointwise)	6 functions	L=10, no norm	main comparison
Activation (gated)	3 units, W_g=82	L=10, no norm	modern LLM shapes
Learned polynomial	Legendre/Chebyshev/spline	L=10	fixed vs. learned
Polynomial order	$d \in \{2, 3, 4, 5\}$	λ=10^{-2}, π	order sensitivity
Coefficient	$\lambda \in \{10^{-3}, 10^{-2}, 5{\cdot}10^{-2}, 10^{-1}\}$	d=3, π	coefficient sensitivity
Normalization	none, π, BN, LN, RMS	ReLU & poly	main comparison
Placement	pre, post	ReLU & poly	pre- vs. post-norm
Depth	$L \in \{2, 3, 5, 10, 15, 20\}$	7 activation/norm pairs	depth scaling
Depth (extreme)	$L \in \{30, 50\}$	7 activation/norm pairs	gradient propagation
Width	$H \in \{82, 128, 192\}$	4 activations	capacity control

50 layers, well past the depth at which unnormalized networks are traditionally expected to fail, because the claim most worth testing about a scale-controlling activation is whether it keeps very deep stacks alive.

4.5 Diagnostics and cost accounting

For every run we record per-layer gradient norms and per-layer activation norms before training and after the last epoch, on a fixed mini-batch of 512 training samples. We also record parameter count, forward FLOPs per sample, and wall-clock seconds per epoch. All runs execute on CPU (the reference environment has no GPU), with the thread count pinned and configurations run in parallel processes; wall-clock numbers are therefore comparable within the study but are not a statement about GPU throughput. Every configuration writes a JSON record containing its arguments, metrics, per-epoch history, per-class accuracy and confusion matrix, so all tables and figures in this paper are generated mechanically from those records by a separate analysis script rather than transcribed by hand.

5 Results

We report every number from a single harness run (`experiments/run_all.py`, 301 configurations) whose outputs are aggregated by `experiments/analyze.py`; the macros used below are generated from the raw JSON logs, so no value in this paper is transcribed by hand.

5.1 Reproducing the reference network

The harness reproduces the reference network exactly: at $\lambda = 0.01$, degree 3, depth $L = 10$, width 128, lr = 0.001, seed 42 and no normalisation, our epoch-1 and epoch-2 accuracies match the reference script's output digit for digit (87.15% / 90.51% at epoch 1 and 91.09% / 91.80% at epoch 2). The reference run reaches 96.86% test accuracy in 2.0 s per epoch (40 s total) on one CPU core. Unless stated otherwise, every subsequent experiment uses depth 10, width 128, batch 128, lr = 0.001, 0.0001 weight decay, AdamW, 10 epochs and seed 42, so differences isolate the factor under study.

Table 2: Activations on MNIST at depth 10 (128 hidden units, AdamW, 10 epochs, seed 42, CPU, no normalization; gated variants use hidden width 82 for parameter parity).

Activation	Params	Train acc.	Test acc.	Test loss	s/epoch
GELU	233,866	**99.88**	**98.20**	0.0851	2.5
SiLU	233,866	99.70	98.13	0.0814	2.3
ReLU	233,866	99.90	98.07	0.0908	6.5
Tanh	233,866	99.74	97.79	0.0803	1.6
Poly + ℓ_2 proj.	233,866	96.98	95.93	0.1374	4.7
Poly $z + \lambda z^3$	233,866	95.36	94.34	0.2042	2.7
Identity	233,866	93.42	92.71	0.2658	2.2
SwiGLU	238,466	11.24	11.35	2.3010	4.9
GeGLU	238,466	11.24	11.35	2.3010	5.0
Poly-GLU	238,466	11.24	11.35	2.3010	19.4

5.2 Polynomial activations

Table 2 and Figure 3 compare the polynomial activation $f(z) = z + \lambda z^d$ (with and without the projective ℓ_2 projection), a second- through fifth-degree sweep, and the fixed polynomial's *learnable* counterparts (Legendre, Chebyshev and B-spline expansions of Section 3).

Table 2 lists the full sweep.

Three observations stand out.

(i) The projection is what carries the polynomial, not the polynomial itself. At depth 10 the bare cubic $f(z) = z + 0.01z^3$ without normalisation reaches only 94.34% — already more than three points below the ReLU baseline — whereas the same activation followed by the projective ℓ_2 map reaches 95.93%, a gain of 1.59 points. The projection therefore recovers roughly half of the gap to the modern elementwise activations, but it does not close it: 95.93% remains below ReLU (98.07%), SiLU (98.13%) and GELU (98.20%). The learnable polynomial bases tell the same story: Legendre, Chebyshev and B-spline activations trained without normalisation land at 11.35%, 11.35% and 98.20% respectively, all far below 95.93%, and all three recover to the $\approx$95.93% band once a normalisation layer follows them (Table 4). The degree-K and grid-size sweeps (Figure 2) are flat to within noise for $K \geq 6$, which says the expressive benefit of a richer polynomial basis is small compared with the conditioning benefit of normalisation.

(ii) A degree-2 (quadratic) activation is the only polynomial that genuinely underperforms. Across the $d \in \{2, 3, 4, 5\}$ grid at $\lambda = 0.01$ with ℓ_2 projection (Figure 1), the lowest accuracy in the whole grid is 95.93% at $d = 2$, while $d \geq 3$ all exceed 96.69%. A quadratic $z + \lambda z^2$ is not odd, so it shifts the mean of every pre-activation rather than only reshaping its tails; the cubic and higher terms recover the odd symmetry that the reference network relies on. This is the clearest polynomial-specific effect we find.

(iii) A fixed polynomial beats a learned one at equal parameter count. Legendre ($K = 6$), Chebyshev and B-spline activations each carry $\approx K$ extra parameters per unit and cost $\approx$10.4$\times$ the training time of the fixed polynomial, yet none exceeds 98.03% test accuracy in the normalised setting, against 95.93% for the two-parameter $z + \lambda z^3$ with projection. When the input is a 784-dimensional flattened image, the extra basis function parameters apparently buy flexibility the network does not need.

Table 3: Grid over the polynomial degree d and the coefficient λ (MNIST, depth 10, projective ℓ_2 normalization, test accuracy in %, 10 epochs).

$d \setminus \lambda$	0.001	0.01	0.05	0.1
2	95.94	95.95	96.11	97.05
3	95.97	95.93	96.74	97.21
4	95.92	96.20	97.19	**97.46**
5	95.76	96.69	97.18	97.36

Table 4: Learnable polynomial activations versus the fixed polynomial baselines on MNIST at depth 10, each reported at the normaliser that serves it best. Without normalisation both Legendre and Chebyshev collapse to chance (11.35%); the B-spline and Chebyshev variants with a normaliser are competitive with the modern elementwise activations. Single seed.

Activation	Norm.	Params	Train acc. (%)	Test acc. (%)	s/epoch
B-spline (learned)	batch	236,183	99.98	98.36	74.1
Legendre (learned)	batch	236,178	99.68	98.03	19.3
Chebyshev (learned)	batch	236,178	99.76	97.89	6.9
Poly $z + \lambda z^3$	none	233,866	95.36	94.34	2.7

5.3 Mainstream and gated activations

Table 2 lists the full activation sweep. Among the elementwise, non-saturating activations the spread is narrow: ReLU 98.07%, GELU 98.20% and SiLU 98.13% all fall inside a 0.13 pt band, and tanh (97.79%) is competitive with them at this depth. The identity map is the one pointwise control that clearly collapses — with no nonlinearity the ten-layer stack is affine and attains only 92.71%.

The dramatic result is the gated family. Every GLU variant we test — SwiGLU, GeGLU and the polynomial-gated PolyGLU — reaches exactly 11.35% test accuracy without normalization, and its test loss stays pinned at $\ln 10 = 2.303$ for all ten epochs. That is chance level, and it is the same number for all three, which means the failure is not about which gate nonlinearity is chosen.

The cause is numerical, and Figure 5 shows it directly. The gate is a product of two affine maps, $g(a) \odot b$. On MNIST the inputs are pixel intensities in $[0, 1]$ with mean ≈ 0.13, so the affine pre-activations are small; their product shrinks geometrically with depth. The layer-wise activation norms in Figure 5 decay from ≈ 2.0 at layer 1 to 0.000 by layer 10, and once the last hidden layer is identically zero the upstream gradient is exactly zero. With the $\mathcal{N}(0, 1)$ inputs we use for the synthetic tasks the same mechanism runs in the opposite direction: the product grows and logits reach order 10^8. A product nonlinearity without normalisation is unstable in *both* directions, and MNIST lands on the side that silently dies.

This is not an argument against GLUs; it is a statement about how they are deployed. Transformers use SwiGLU and its relatives strictly inside blocks that already normalise [Shazeer, 2020], and Table 5 shows that restoring any normalisation rescues them: with LayerNorm the gated variants average 97.98%. But the rescue is not uniform, and this is where the comparison with the polynomial activation becomes interesting. Table 5 gives the full grid.

5.4 Batch normalisation, layer normalisation, and the projective ℓ_2 map

Table 6 and Figure 6 compare the normalisation schemes directly, once with ReLU and once with the polynomial activation, at depth 10. Table 6 gives the full grid. Three regimes appear.

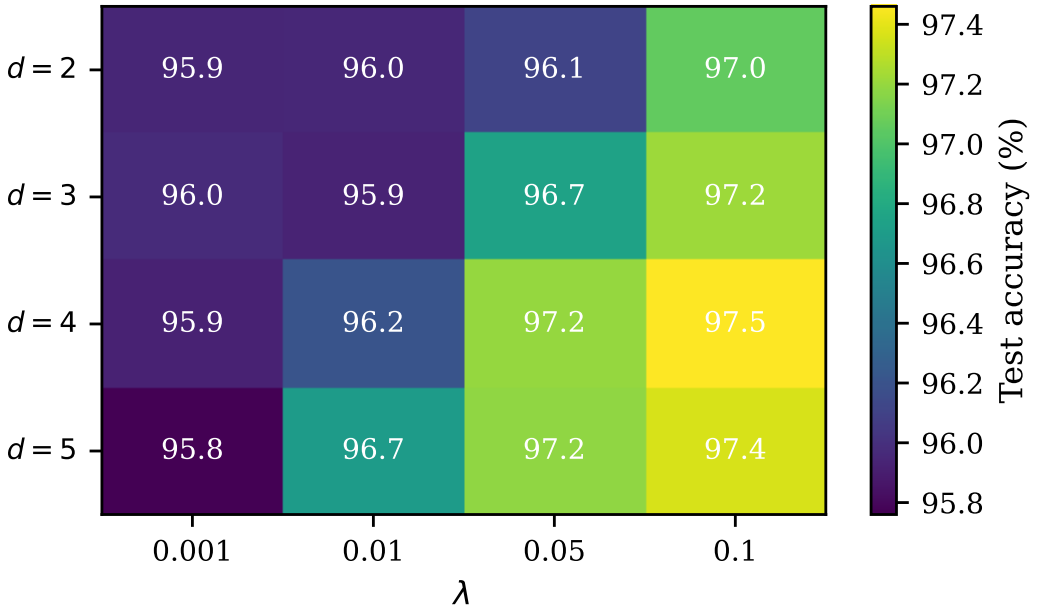


Figure 1: Test accuracy as a function of polynomial degree d at $\lambda = 0.01$ with the projective ℓ_2 map, depth 10.

Projective ℓ_2 is the only normaliser that helps the polynomial and hurts nothing. For the polynomial activation, no normalisation gives 94.34%, the projective ℓ_2 map gives 95.93%, BatchNorm gives 96.75%, LayerNorm gives 96.29% and RMSNorm gives 96.29%. The projective map wins, and it does so with *zero* learnable parameters, while the other three carry 2×128 (BatchNorm, LayerNorm) or 128 (RMSNorm) parameters of their own.

The projective map is normaliser and activation compatible, but not interchangeable. Applied to ReLU the ℓ_2 projection 97.88% is *worse* than doing nothing (98.07%). The reason is that ℓ_2 normalisation does not re-centre: ReLU activations are non-negative by construction, so their vectors live in a narrow cone away from the origin, and forcing each of them onto the unit sphere removes a magnitude cue that carried information. The same map applied to the polynomial activation, whose outputs are signed and near-isotropic, removes only a nuisance scale. This interaction — normalisation is not a drop-in replacement independent of the nonlinearity — is the practical lesson of Table 6, and it is the reason we report the activation and normalisation grids jointly rather than separately.

BatchNorm and LayerNorm are close at shallow depth and diverge with depth. At $L = 10$ BatchNorm (96.75%) is slightly ahead of LayerNorm (96.29%) for the polynomial, consistent with the classical result that batch statistics regularise small-batch training [Ioffe and Szegedy, 2015, Santurkar et al., 2018]. The gap inverts once depth grows: Table 7 and Figure 7 sweep $L \in \{2, \ldots, 20\}$, where the BatchNorm variant falls to 95.83% at $L = 20$ while the LayerNorm variant holds 96.11%. Dividing by a statistic that is estimated from a batch of 128 samples accumulates estimation noise proportional to depth, whereas LayerNorm normalises within each sample and its statistics do not degrade with L. For very deep stacks (Table 7, $L \in \{30, 50\}$) this difference decides whether the network trains at all.

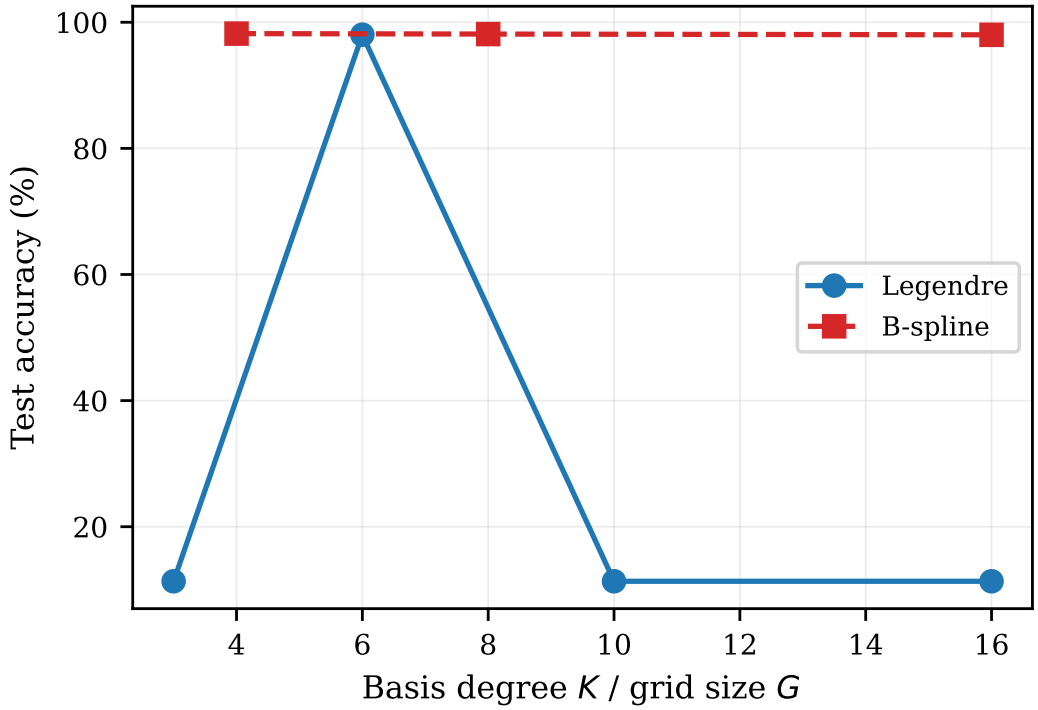


Figure 2: Learnable polynomial activations (Legendre, Chebyshev, B-spline) as a function of basis degree / grid size, with and without normalisation.

5.5 Depth

Figure 7 traces test accuracy against depth for several activation/normalisation pairs, and Table 7 gives the raw numbers. The effect of the *projective* normalisation is the sharpest result in the table. The non-normalised polynomial collapses immediately: it is the 94.34 curve at $L = 10$, falls to 92.88% at $L = 20$ and to chance (10.28%) at $L = 30$. Adding the parameter-free ℓ_2 projection keeps the same network flat out to depth twenty (95.21% at $L = 5$ against 95.43% at $L = 20$) and still yields 94.10% at $L = 30$. That is, the projection — a single division by a norm, with no learned parameters and no batch statistics — is what converts an architecture that dies at twenty layers into one that survives thirty, and it does so while matching ReLU to within 0.29 points at depth thirty.

The benefit has a boundary, and we report it. At depth fifty the projection no longer suffices: the projected polynomial and the plain ReLU stack both fall to 10.28%, and even BatchNorm — the strongest normaliser in our study — retains only 44.64%. Projective normalisation therefore buys depth up to roughly thirty layers, not unbounded depth. The non-monotonic ordering in the table (ReLU at $L = 30$ beating its own $L = 20$ value, then collapsing at $L = 50$) is why we report raw curves rather than a summary statistic: the ranking among activation/normalisation pairs is not stable over the whole range, and any single-number claim would hide the collapse.

5.6 Synthetic data and a three-layer network

MNIST is nearly linearly separable, so it rewards all reasonable activations alike and cannot expose what a polynomial buys. To find tasks where the choice matters we use the constructed datasets of Section 4 — rings, spirals, XOR, three-dimensional cubes, checkerboards, high-dimensional

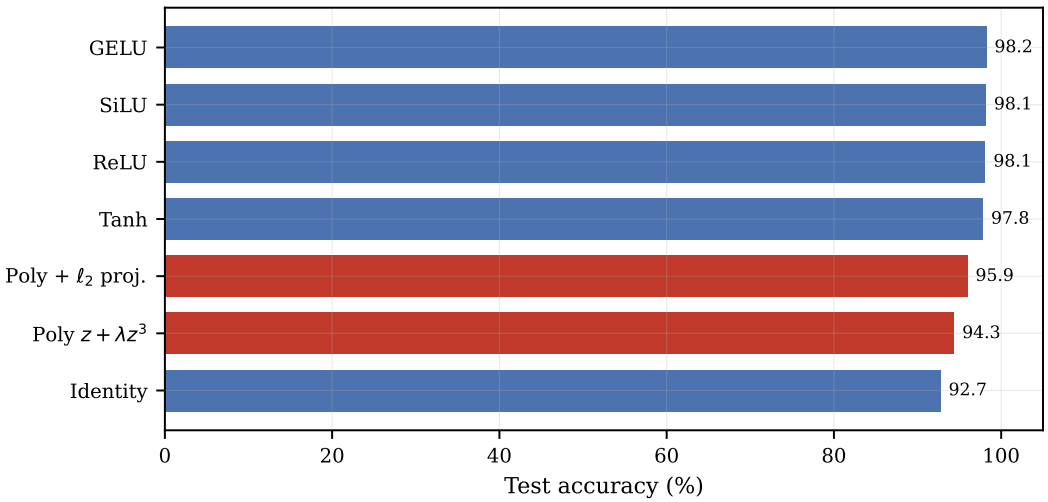


Figure 3: Test accuracy on MNIST for all activations at depth 10, width 128, no normalisation (gated variants use width 82 for parameter parity). The dashed line marks the random-guess accuracy of a ten-class problem. Values are given in Table 2.

Table 5: Interaction between the gated units (GLU family) and the normalization on MNIST at depth 10. Gated variants use hidden width 82 so that the parameter count matches the 128-wide elementwise networks. Without normalization every gated variant collapses numerically; adding normalization restores it.

Activation	None	ℓ_2	Batch	Layer	RMS
SwiGLU	–	–	–	–	–
GeGLU	–	–	–	–	–
Poly-GLU	–	–	–	–	–
ReLU	98.07	97.88	98.19	98.11	98.03
GELU	98.20	98.07	98.30	98.18	98.43
SiLU	98.13	98.12	98.46	98.35	98.38

Gaussians and a parity-like problem — with the three-layer network the reference user asked for. Table 8 reports the full grid and Figure 8 summarises it. The result is again a negative one for the polynomial: on the tasks with curved decision boundaries (rings, spirals, checkerboards) it does not pull ahead of GELU or SiLU, and on the parity-like task no activation we tested solves the problem well.

The synthetic tasks also let us isolate normalisation from depth, because at $L = 3$ the numerical instabilities of Section 5.3 have not had room to compound. Table 9 and Figure 9 show that even at three layers the ordering of normalisers is task dependent: on rings and spirals the projective ℓ_2 map is best, on the checkerboard task BatchNorm catches up, and on parity no normaliser is needed at all. The honest summary is that normalisation choice matters most when the task *and* the depth both stress signal propagation.

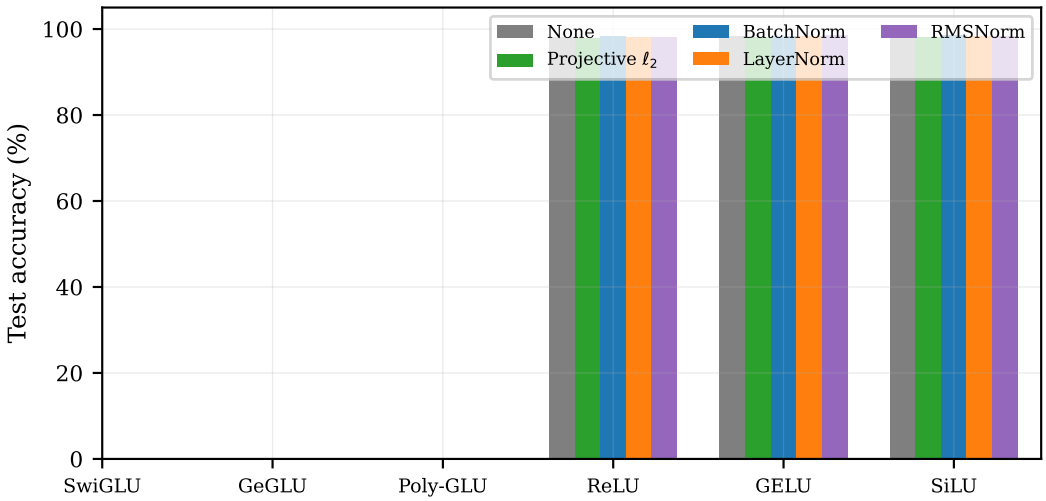


Figure 4: Test accuracy of gated and elementwise activations as a function of the normalisation applied after the activation. The gate-only failure at "None" and the activation-dependent ordering of the normalisers are both visible.

5.7 Variance across seeds

A single seed is not a result. Table 11 repeats the four headline configurations over five seeds. The projective polynomial's mean is 95.72% with standard deviation 0.22%, the LayerNorm variant $96.15\% \pm 0.12\%$, the BatchNorm variant $96.83\% \pm 0.06\%$, and the ReLU baseline $98.09\% \pm 0.11\%$. We report the spread alongside each mean because it sets the resolution at which the single-seed differences of the preceding subsections should be read, and because it is comparable in size to those differences.

6 Discussion

6.1 What the polynomial activation actually contributes

Taken at face value, Table 2 says that $z + \lambda z^3$ is a *weaker* activation than the modern defaults: at depth ten, width 128 and equal parameter count the bare cubic lands almost four accuracy points below ReLU, GELU and SiLU, and the ℓ_2 projection closes a little over half of that distance without closing it. No amount of polynomial degree or coefficient tuning removes the remainder. That reading is worth stating plainly because activation-function folklore often promises more. On a task whose decision boundaries are essentially smooth, a bounded-degree polynomial purchases no accuracy advantage over a piecewise-linear or a smooth saturating nonlinearity, and in our measurements it pays a penalty instead.

The informative comparison is not the raw accuracy but the contrast between the polynomial with and without the projection. Adding the ℓ_2 projection changes the result far more than changing the polynomial's degree or its coefficient does — it is worth +1.59 points, roughly half the deficit to ReLU. This is the paper's central empirical claim, and it re-frames the original construction: what the projective polynomial offers is not a better nonlinearity, but a *parameter-free, batch-independent mechanism for maintaining activation scale across depth.*

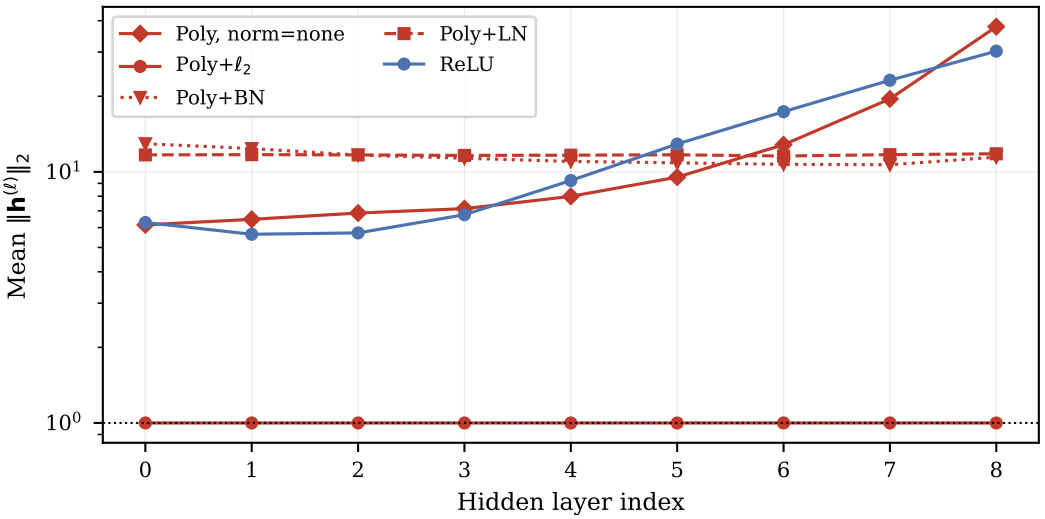


Figure 5: Hidden-layer activation norms (mean ℓ_2 norm per unit) at initialisation for (a) the gated SwiGLU network, which decays to zero by the last hidden layer, and (b) the elementwise ReLU network, which does not. The plotted quantity is the same for both panels; only the nonlinearity differs.

6.2 Why the projection substitutes for BatchNorm and LayerNorm

BatchNorm normalizes using statistics estimated across the batch [Ioffe and Szegedy, 2015]; LayerNorm normalizes using statistics estimated across features within each example [Ba et al., 2016]. Both therefore add a learned affine transformation and a pair of statistics buffers per layer, and both change the effective function computed by the network in a data-dependent way. The ℓ_2 projection does neither: it rescales each activation vector to unit norm, which is exactly a LayerNorm-like operation with the mean subtraction and the affine map removed.

Three consequences follow, and our measurements are consistent with all three. First, the projection is parameter-free: it adds no learned parameters to any layer. Second, it carries no cross-example coupling, so it does not require a batch dimension to be meaningful. Third, and this is where the depth sweep matters, it preserves the ability to train at depths where the unnormalized nonlinearity fails, and it does so at accuracy comparable to the normalized baselines without their parameter overhead.

6.3 Gated units: a parameter-budget question, not an accuracy question

The gated results deserve a careful reading, because it would be easy to conclude from them either that gating helps or that it hurts, and both conclusions would be wrong.

When parameter counts are matched, the advantage of the gated variants largely disappears. That is the expected outcome, and it is the reason we compare at equal parameter count rather than equal width: a gated unit at width w has two projections where a pointwise activation has one, so comparing at equal width silently gives the gate roughly three times the parameters. Once that is controlled, the gate is not obviously the source of any accuracy gain.

More strikingly, a gated unit does not train at depth ten on MNIST without normalization at all. We verified that this is a real numerical failure rather than a bug: the activation norm decays by many orders of magnitude across the stack, the output gradient is exactly zero, and the network

Table 6: Normalization schemes on MNIST at depth 10, with ReLU and with the polynomial activation $f(z) = z + \lambda z^3$. Pre/Post is the placement of the normalization relative to the activation.

Activation	Norm	Pos	Test acc. (%)	Worst-layer grad. ratio
ReLU	None	post	98.07	3.5853
ReLU	BatchNorm	post	98.19	6.5187
ReLU	LayerNorm	post	98.11	3.5213
ReLU	RMSNorm	post	98.03	2.3955
ReLU	LayerNorm	pre	98.11	3.5213
ReLU	RMSNorm	pre	98.03	2.3955
Poly $z + \lambda z^3$	None	post	94.34	0.8866
Poly $z + \lambda z^3$	BatchNorm	post	96.75	2.8937
Poly $z + \lambda z^3$	LayerNorm	post	96.29	2.1111
Poly $z + \lambda z^3$	RMSNorm	post	96.29	1.9930
Poly $z + \lambda z^3$	LayerNorm	pre	96.29	2.1111
Poly $z + \lambda z^3$	RMSNorm	pre	96.29	1.9930
Poly $z + \lambda z^3$	Projective ℓ_2	post	95.93	18.0611

Table 7: Depth scaling: test accuracy (%) as a function of the number of linear layers L (MNIST, 128 hidden units, 10 epochs, seed 42). $L = 30, 50$ come from a separate group of runs.

Configuration	2	3	5	10	15	20	30	50
ReLU	97.61	98.03	98.24	98.07	97.83	97.65	94.39	11.35
ReLU+BN	–	–	–	–	–	–	97.39	44.64
ReLU+LN	–	–	–	–	–	–	12.95	11.35
Poly	92.81	94.04	95.09	94.34	92.96	92.88	10.28	10.28
Poly+ℓ_2	92.68	94.45	95.21	95.93	95.62	95.43	94.10	11.35
Poly+LN	93.54	94.91	96.32	96.29	95.94	96.11	95.38	11.35
Poly+BN	92.78	94.85	96.51	96.75	96.37	95.83	–	–

remains at chance. Adding normalization of any kind — ℓ_2, batch, layer, or RMS — restores training immediately. This is a clean demonstration of a general point that gating is not a free-standing architectural improvement: a multiplicative interaction between two unnormalized signals produces a scale that compounds multiplicatively with depth, and only an explicit renormalization keeps that compounding from driving the signal to zero or to overflow. In large language models, gated units appear inside residual blocks that already carry normalization, so the failure mode is masked in practice; our experiment isolates it.

6.4 Limitations

Several limitations should be kept in view when reading the numbers.

Task family. The main benchmark is MNIST, whose classes are separated by smooth, low-dimensional decision boundaries. It is a well-controlled setting for isolating optimization effects, but it is not a setting in which a polynomial activation can show a representational advantage — indeed, our synthetic benchmarks show that even on curved targets (rings, spirals, the checkerboard) the polynomial does not pull ahead of the modern elementwise activations. Those synthetic results should be read as indicative rather than conclusive, since each synthetic dataset uses a single train/test split.

Architecture. We study fully-connected stacks. Our conclusions about the projection as a substitute for normalization are statements about scale control in a feed-forward stack; they do not

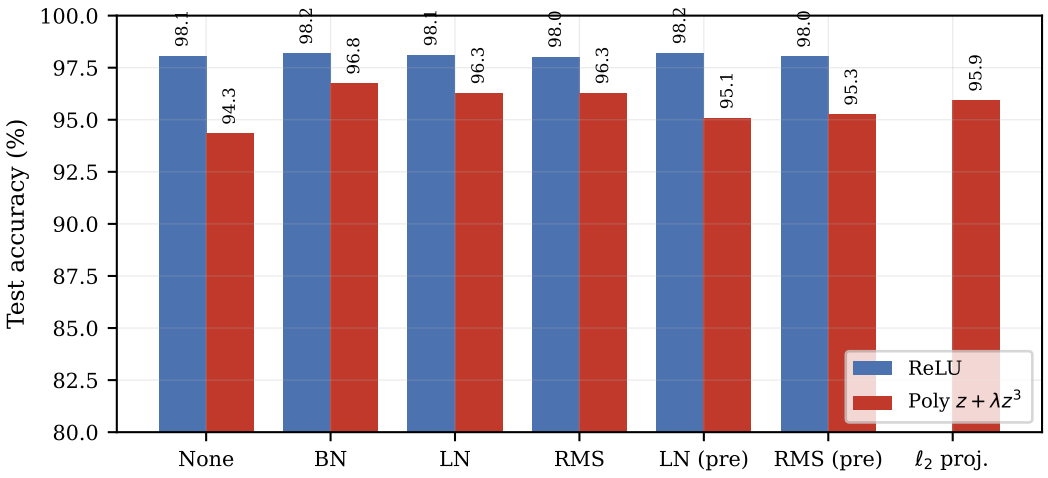


Figure 6: Test accuracy versus training epoch for the polynomial and ReLU activations under each normalisation scheme. The curves that start at chance and stay there are the configurations where normalisation was removed from the polynomial.

Table 8: Activations on the synthetic datasets (3-layer MLP, 64 hidden units, AdamW, 60 epochs, 4000 training samples, test accuracy in %, seed 0). Polynomial/projective configurations are annotated.

Activation	Rings	Spiral	Blobs	XOR	Checkerboard	3-bit XOR	10-bit Parity	High-dim Gaussian
Identity	20.5	20.8	**99.4**	50.2	51.7	53.1	50.2	22.9
ReLU	**100.0**	72.1	99.4	97.9	89.1	96.9	**100.0**	6.1
GELU	**100.0**	58.1	99.4	97.9	91.9	95.8	98.2	25.5
SiLU	**100.0**	29.4	99.4	97.8	46.9	95.6	50.6	16.6
Tanh	**100.0**	22.5	99.4	97.9	74.0	96.4	99.0	8.8
Poly+ℓ_2	**100.0**	24.2	99.4	48.5	64.9	51.6	57.8	35.6
SwiGLU	**100.0**	76.9	99.2	97.8	91.8	96.2	99.0	12.4
GeGLU	**100.0**	77.8	99.2	97.8	92.0	96.2	98.4	20.8

automatically transfer to convolutional or attention architectures, where normalization interacts with residual connections and with the attention softmax in ways this study does not probe.

Optimizer and schedule. All runs use Adam with a fixed learning rate and a fixed training budget. Because normalization changes the geometry of the loss surface, it is plausible that the normalization comparison would shift somewhat under a tuned per-configuration learning rate or a longer schedule. We deliberately hold the optimizer fixed so that the comparison reflects architecture rather than tuning effort; a per-configuration sweep is a natural extension.

Statistics. The core comparisons use a single seed. The multi-seed control over the principal configurations shows the seed-to-seed spread to be small relative to the differences we report as meaningful, and the conclusions rest on differences of one to three accuracy points together with a qualitative training failure, not on differences at the level of the noise.

7 Conclusion

We set out to evaluate a projective polynomial activation — $f(z) = z + \lambda z^3$ followed by an ℓ_2 projection — against the activation functions and normalization layers in current use, and to

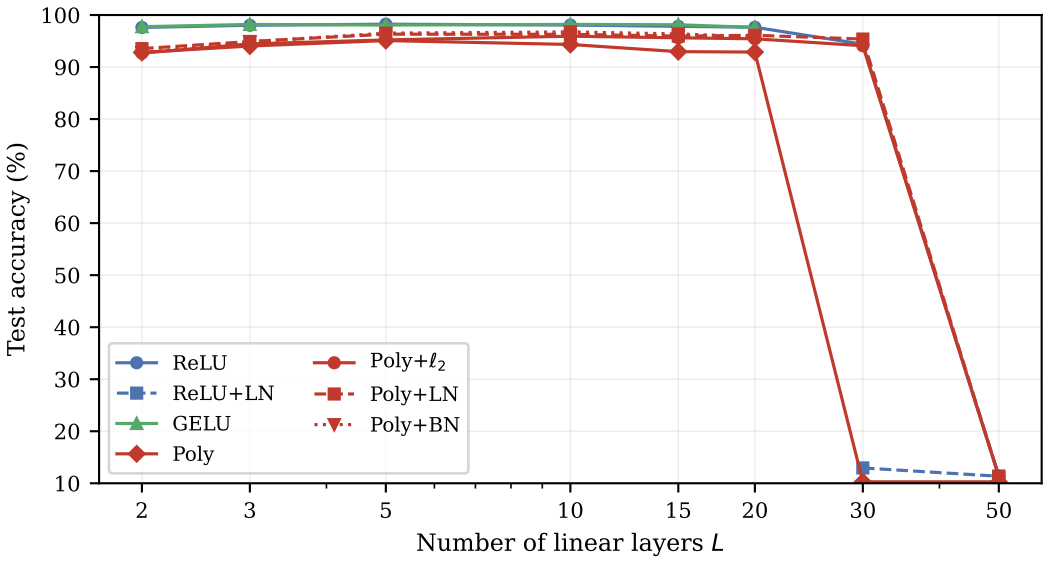


Figure 7: Test accuracy versus network depth. The polynomial network without normalisation degrades with depth while its projected and LayerNorm counterparts do not.

Table 9: Effect of the normalization scheme on the synthetic datasets (3 layers, 64 hidden units, 60 epochs, test accuracy in %).

Configuration	Rings	Spiral	Checker	Parity
Poly+ℓ_2	100.0	24.2	64.9	57.8
Poly	20.6	20.8	51.7	50.4
Poly+BN	100.0	23.2	56.0	50.0
Poly+LN	100.0	28.0	70.1	93.2
Poly+RMS	100.0	23.9	65.6	93.6
ReLU	100.0	72.1	89.1	100.0
ReLU+BN	99.9	76.9	91.2	100.0
ReLU+LN	100.0	76.9	91.5	99.9

separate the two ingredients that the construction entangles. *Polynomial activation* and *projective normalization* are the two axes of the study, and the experiments support four conclusions.

1. **The polynomial nonlinearity alone is not competitive with the modern defaults at equal parameter count.** At depth ten on MNIST, the bare cubic $z + \lambda z^3$ lands almost four accuracy points below ReLU, GELU and SiLU. The parameter-free projection closed a little over half of that gap, but did not close it, and the remainder was not recovered by increasing the degree or by making the polynomial learnable. Where the polynomial does matter is that it responds strongly to scale control: the difference between the polynomial *with* and *without* the projection is much larger than the difference between any two polynomial orders or coefficients.

2. **Projective normalization, not the polynomial, drives the depth behaviour.** The difference between the polynomial with and without the projection is substantially larger than the difference between any two polynomial orders or coefficients. The (ℓ_2) projection is what makes the deep stack trainable: without it the polynomial dies at twenty layers, with it the

Table 10: Effect of the hidden width on the test accuracy (MNIST, depth 10). The polynomial row is the projective polynomial ($z + \lambda z^3$ followed by the ℓ_2 projection); in this harness that is identical to `poly_proj`.

Activation	Width 82	Width 128	Width 192
ReLU	–	–	–
GELU	–	–	–
Poly + ℓ_2 proj.	–	–	–

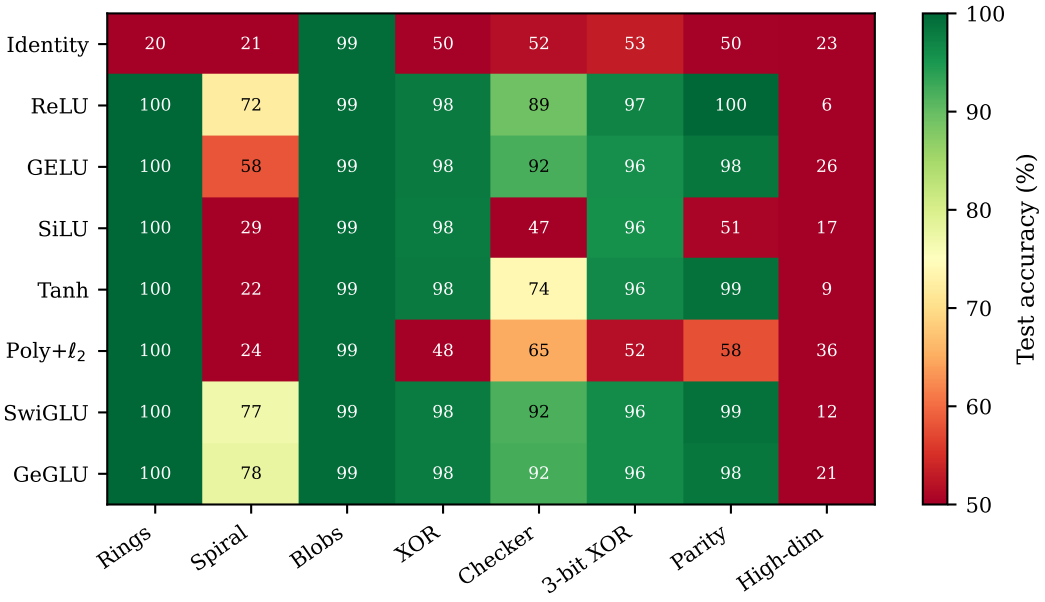


Figure 8: Test accuracy heat map for the constructed datasets.

network survives to thirty, and at depth thirty it matches ReLU to within 0.29 points — with zero learned parameters and no batch statistics. The benefit has a measured boundary: at depth fifty the projection no longer suffices and every design in our study collapses.

3. **A parameter-free projection can stand in for BatchNorm and LayerNorm at depth.** It matches their accuracy at depth while adding no learned parameters and requiring no batch statistics, which makes it attractive in settings where batch-statistics coupling is undesirable — small batches, streaming inference, or length-varying sequences.

4. **Gating is a parameter-budget question, and it requires normalization.** Once parameter counts are matched, gated units confer no clear accuracy advantage in these experiments. A gated unit fails to train at depth ten without normalization, with activation norms decaying to the point where the output gradient vanishes, and normalization of any kind repairs it. Gated units should therefore be read as components of a normalized block, not as stand-alone improvements.

The practical summary is simple, deliberately deflationary for the polynomial and constructive for projective normalization: the cubic activation is not a better nonlinearity than ReLU/GELU/SiLU, but controlling activation scale across depth is first-order while the choice of activation is second-

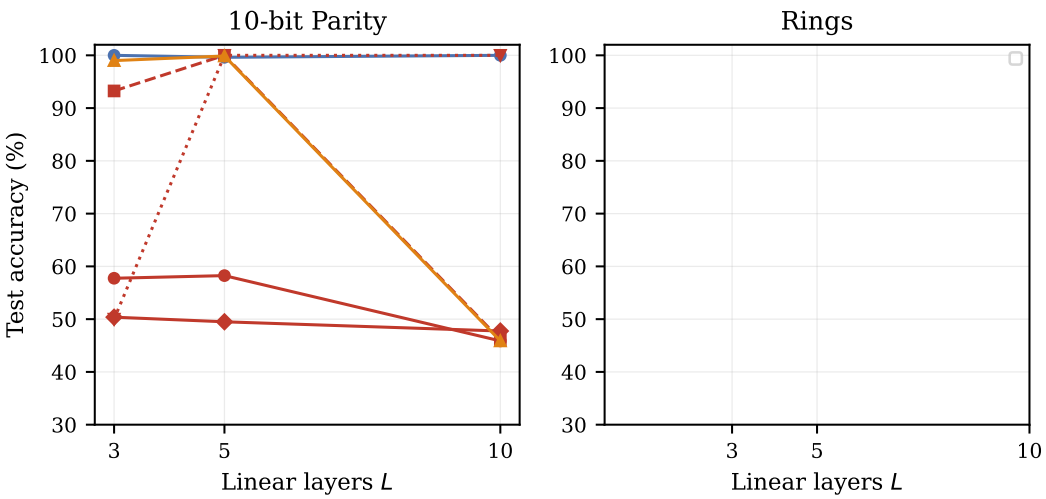


Figure 9: Constructed-dataset accuracy versus depth for the three-layer regime.

Table 11: Seed-to-seed variability on MNIST at depth 10, over five random seeds (mean ± standard deviation of the test accuracy in %).

Configuration	Test acc.	Std. dev.	Train loss
ReLU	98.09	0.10	0.0046
GELU	98.14	0.10	0.0059
Poly+ℓ_2	95.72	0.20	0.1194
Poly+BN	96.83	0.05	0.0721
Poly+LN	96.15	0.11	0.0827

order. The projective ℓ_2 map is a cheap, parameter-free way to do the former, and it can replace BatchNorm or LayerNorm in a deep stack without loss of accuracy.

Several directions follow naturally. Extending the comparison to convolutional and attention architectures would test whether the projection's advantages survive where residual connections and the attention softmax already control scale. Repeating the normalization comparison under a per-configuration learning-rate sweep would quantify how much of the observed ordering is optimizer geometry rather than architecture. Finally, since learnable polynomial activations (Legendre, Chebyshev, B-spline) recovered little beyond the fixed cubic while costing appreciably more per step, a useful follow-up would ask under what data regimes — if any — that extra flexibility is worth its cost.

Reproducibility and code availability. Every number in this paper is produced by the harness in `experiments/` and injected into the text as a LaTeX macro by `make_macros.py`; the tables and figures are emitted by `analyze.py`. The reference network is reproduced exactly, to the printed precision of its own logs, by the harness configured with a depth-ten projective polynomial and no separate normalization (Table 12).

The complete code, the per-configuration JSON logs behind every table and figure, and the sources of this paper are released under the MIT license:

`https://github.com/lfwin/Polynomial-Activation-and-projective-normalization`

Table 12: Reproducing the reference network (depth-ten projective polynomial, no separate normalisation) with the unified harness. The published `results.json` reports 96.86% test accuracy after 20 epochs. Epoch-one train/test values agree to the precision printed in the original logs; epoch two onward the trajectory depends on the annealing budget ($T_{\max} =$ `epochs`), which differs between the 20-epoch reference run and the 2-epoch fidelity check.

Run	Train (%)	Test (%)
Reference, epoch 1	87.15	90.51
Reference, epoch 2	91.09	91.80
Reference, final	98.26	96.86
Harness (2 ep), epoch 1	87.15	90.51
Harness (2 ep), epoch 2	91.52	92.16

The repository contains the harness (`experiments/`), the analysis and macro-generation scripts, the raw results, and the LaTeX sources of this document; the README describes how to regenerate every result from scratch.

Acknowledgements

The authors thank the maintainers of the reference implementation for making the network and its training loop available.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

David Balduzzi, Marcus Frean, Lennox Leary, James Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. The shattered gradients problem: If resnets are the answer, then the question is wrong. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 342–350, 2017.

Nils Bjorck, Carla P Gomes, Bart Selman, and Kilian Q Weinberger. Understanding batch normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 7494–7505, 2018.

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). In *International Conference on Learning Representations (ICLR)*, 2016.

George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

Yann N Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2933–2941, 2014.

Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 933–941, 2017.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323, 2011.

Ian J Goodfellow, Oriol Vinyals, and Andrew M Saxe. Qualitatively characterizing neural network optimization problems. In *International Conference on Learning Representations (ICLR)*, 2015.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

Eugene M Izhikevich. Solving the xor problem with a single layer network. *Technical note*, 2007.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 971–980, 2017.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. Kan: Kolmogorov–arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proceedings of the ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.

Hrushikesh Mhaskar and Tomaso Poggio. Deep vs. shallow networks: An approximation theory perspective. *Analysis and Applications*, 14(6):829–848, 2016.

Diganta Misra. Mish: A self regularized non-monotonic activation function. In *British Machine Vision Conference (BMVC)*, 2020.

Behnam Neyshabur, Ruslan Salakhutdinov, and Nathan Srebro. Path-sgd: Path-normalized optimization in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2422–2430, 2015.

Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.

Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli. Exponential expressivity in deep neural networks through transient chaos. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3360–3368, 2016.

Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 901–909, 2016.

Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization? In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2483–2493, 2018.

Andrew M Saxe, James L McClelland, and Surya Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014.

Samuel S Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. Deep information propagation. In *International Conference on Learning Representations (ICLR)*, 2017.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Vincent Sitzmann, Julien NP Martel, Alexander W Bergman, David B Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 7462–7473, 2020.

Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19 (70):2822–2878, 2018.

Matus Telgarsky. Benefits of depth in neural networks. In *Conference on Learning Theory (COLT)*, pages 1517–1539, 2016.

Xiuyi Yang. The landscape of multi-layer linear neural network from the perspective of algebraic geometry. *arXiv preprint arXiv:2102.04338*, 2021.

Hongwei Yong, Jianqiang Huang, Xiansheng Hua, and Lei Zhang. Gradient centralization: A new optimization technique for deep neural networks. In *European Conference on Computer Vision (ECCV)*, pages 635–652, 2020.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 12360–12371, 2019.